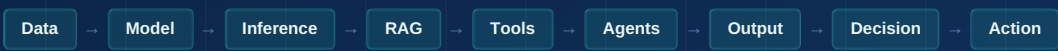


From Data to Decisions

The Complete End-to-End AI Journey

How raw data becomes trained models, models become intelligent systems, and intelligent systems produce outputs, decisions, and business action — one continuous journey across Machine Learning, Foundation Models, Generative AI, Retrieval, Tools, Agents, and Agentic AI.

THE JOURNEY, END TO END



AUDIENCE

CXOs · Technology Leaders · Enterprise
Architects · AI Practitioners

Version 1.0 | September 2026

AUTHOR

Mohan Arumugam

Table of Contents

Fourteen pages, built to be read start to finish or referenced section by section.

FRONT MATTER

—	Executive Summary One continuous system, and five takeaways for CXOs	3
---	--	----------

PART I — THE TECHNICAL MENTAL MODEL

1	The Layered Mental Model Why AI, ML, GenAI, RAG, and agents are layers, not competitors	4
2	How a Foundation Model Gets Built Data, pretraining, and post-training alignment	5
3	From Trained Model to Live Application Inference, context assembly, and where "reasoning" fits	6
4	RAG: Grounding AI in Enterprise Knowledge Retrieval-Augmented Generation, explained and compared	7
5	Tool Use: Connecting AI to Systems of Record From answering questions to checking and triggering systems	8
6	Agents and Agentic AI From answers to actions — the agent loop and multi-agent orchestration	9

PART II — APPLYING THE MODEL

7	One Request, Traced Through the Full Stack A complete enterprise walkthrough, layer by layer	10
8	Governance, Risk, and Control for CXOs The risk table, human-in-the-loop, and why pilots stall	11
9	Strategic Implications and a Practical Roadmap A five-stage maturity model and four recommendations	12

APPENDICES

A	Glossary Every term used in this document, defined in plain language	13
B	References Eleven primary, independently verifiable sources, grouped by topic	14

EXECUTIVE SUMMARY

Executive Summary

One continuous system, not ten competing buzzwords — and why that distinction changes how leadership should fund, sequence, and govern AI investment.

Boards and executive teams are bombarded with AI terminology — machine learning, foundation models, generative AI, RAG, agents, agentic AI — often presented as competing categories or successive hype cycles. They are not. They are **layers of one continuous system** that starts with enterprise data and ends with a business action: a report written, a case resolved, a transaction executed, a decision made.

This guide gives CXOs a working mental model for that system, end to end, without requiring a background in machine learning. It traces a single request through every layer of the stack — from the moment a user or system issues a goal, through model training and inference, retrieval of enterprise knowledge, tool and system calls, to a final validated, human-approved output — and it draws out the strategic, financial, and governance implications at each layer.

WHY THIS MATTERS NOW

Gartner projects that **40% of enterprise applications will embed task-specific AI agents by the end of 2026**, up from under 5% in 2025 — while separately cautioning that **over 40% of agentic AI projects will be cancelled by 2027** due to unclear business value, escalating cost, or inadequate risk controls. The gap between those two numbers is a governance and architecture problem, not a technology problem. This document is written to help close it.

Five things every CXO should take away

1

It is one stack, not ten technologies

AI, ML, deep learning, foundation models, GenAI, RAG, tools, and agents are layers of a single pipeline — data in, decision out — not separate products to buy independently.

2

The model is not the product

A foundation model provides raw capability. The business value is created by the orchestration, retrieval, tools, memory, and governance built around it.

3

Agentic AI raises the stakes

Moving from “the model answers” to “the system acts” multiplies both the value and the risk surface — and demands human-in-the-loop controls by design.

4

Grounding beats retraining — for knowledge

For keeping a model current on proprietary facts, Retrieval-Augmented Generation is far cheaper and safer than retraining. Fine-tuning still has its place — for teaching behavior and style, not facts (Section 4).

5

Governance is the multiplier

The organizations separating pilots that ship from pilots that stall are the ones that treat evaluation, human oversight, and cost control as first-class design requirements, not afterthoughts.

Two journeys, one document

Everything in this document is one of two journeys. **Journey one is how intelligence is created** — data becomes a trained model (Sections 2–3). **Journey two is how intelligence is used** — a business request becomes a validated decision or action, drawing on that model plus retrieval, tools, and agents (Sections 4–9). Confusing the two is the most common source of unrealistic AI timelines: Journey one happens rarely, at capital scale, mostly outside your organization; Journey two happens on every request, and it's where nearly all enterprise engineering effort actually goes.

THE SHAPE OF THE WHOLE DOCUMENT

Data → training → model (Journey one) → **inference → RAG / tools / agents → validation → human oversight → decision → business action** (Journey two) — with governance wrapping both, start to finish, not bolted on at the end. Section 9 closes the loop and returns to this exact chain.

How to use this document

Sections 1–6 build the technical mental model layer by layer, in plain language, each with a diagram and a “what it means for leadership” callout. Section 7 traces one enterprise request end to end. Sections 8–9 turn the model into governance guidance and a strategic roadmap. A glossary and a reference list of primary, verifiable sources close the document.

SECTION 1 · MENTAL MODEL

1. The Layered Mental Model

Stop treating AI terms as competitors. They are layers of one stack — each one built on the layer below it.

Most confusion in AI strategy conversations comes from treating "machine learning," "generative AI," "RAG," and "agents" as alternatives — as if a company must choose one. In practice they occupy **two different dimensions**: a **model layer** (what kind of engine is doing the computation) and an **application/system layer** (what is built on top of that engine to make it useful). Collapsing both dimensions into one straight line is where most AI diagrams — and most AI strategy decks — go wrong.

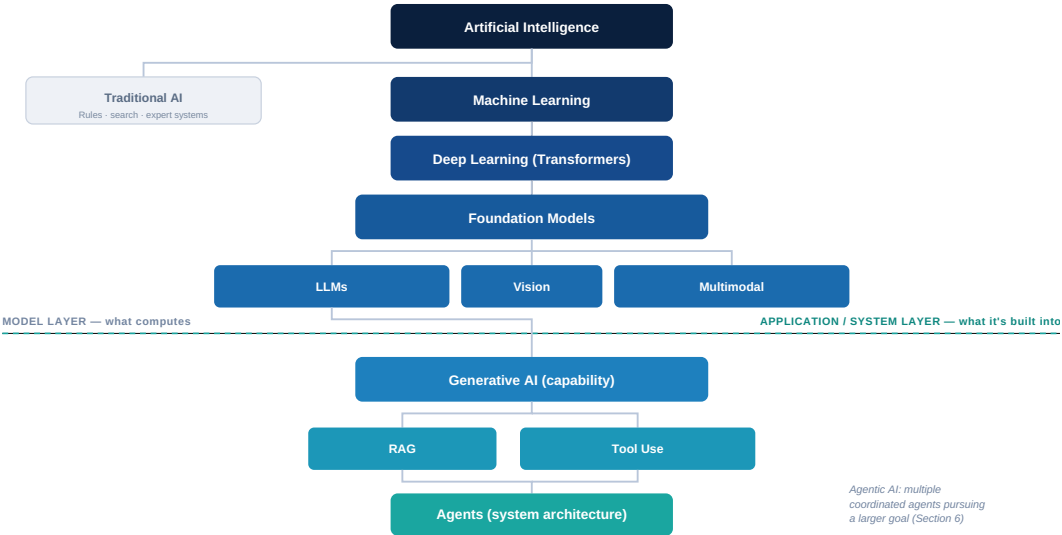


Figure 1. The AI stack is two-dimensional, not one straight line. The **model layer** (top) determines what kind of engine is doing the computing; the **application/system layer** (bottom) determines what that engine is built into. Generative AI is a capability of the model layer's output; RAG, tool use, and agents are system-architecture patterns built around it.

Model layer vs. application layer, precisely

TERM	WHICH LAYER	WHAT IT ACTUALLY IS
Machine learning	Model layer	An approach to building models: learning patterns from data rather than hand-coded rules
Deep learning	Model layer	A subset of ML using layered neural networks; modern LLMs use the Transformer architecture
Foundation model	Model layer	A model paradigm: pretrained at broad scale, adaptable to many downstream tasks
LLM	Model layer	One class of foundation model, specialized for language
Generative AI	Application layer	A capability — producing new content — that applications build using foundation models
RAG	Application layer	An information-retrieval architecture pattern, not a model or a product
Tool use	Application layer	An interface pattern connecting a model to external functions and systems
Agent / Agentic AI	Application layer	A system architecture (agent) or paradigm (agentic AI) orchestrating models, tools, and memory

The three questions that cut through the jargon

Whenever a vendor or team pitches "AI," "GenAI," or "an agent," three separate questions clarify what is actually being proposed:

QUESTION	ANSWER TYPE	EXAMPLE
What is the model?	The underlying engine	A large language model, a vision model, or a multimodal foundation model
What does the model do?	A capability	Prediction, classification, generation, reasoning, or embedding
What system is built around it?	An application or architecture	A chatbot, a RAG-grounded assistant, a copilot, an agent, or a multi-agent workflow

WHAT THIS MEANS FOR LEADERSHIP

When a team says "we're building an AI agent," ask which layer they actually mean. A well-scoped answer names the model, the data it will be grounded on, the tools it will call, and the human checkpoints — not just "we're using GenAI." If a proposal cannot answer all three questions above, it is not yet ready to be funded.

2. How a Foundation Model Gets Built

Before any business ever asks it a question, a model passes through a long, capital-intensive pipeline. Understanding its shape explains both the model's power and its limits.

Section 1 placed the foundation model at the center of the stack. This section opens it up: every foundation model — the engine underneath ChatGPT, Claude, Gemini, or any enterprise copilot — is produced *before* anyone deploys it, in a process completely separate from how it is later used. Leadership teams that understand this lifecycle ask better questions of vendors: which data was this trained on, when was it last updated, and what alignment work was done to make it safe to deploy?

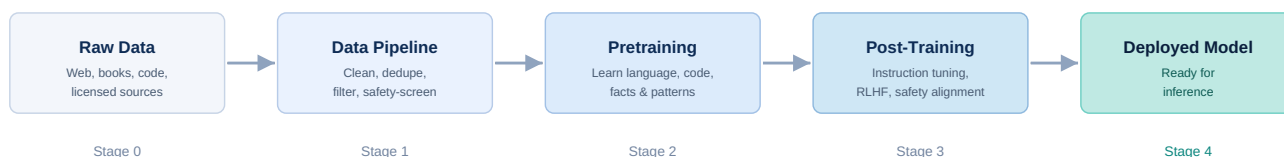


Figure 2. The model-creation pipeline: this entire sequence happens once, ahead of time — independent of any single business question the model will later answer.

What actually happens inside one training step

"The model learns patterns" is true but abstract. Concretely, every single training step repeats one mechanical loop, executed billions of times over the training run:

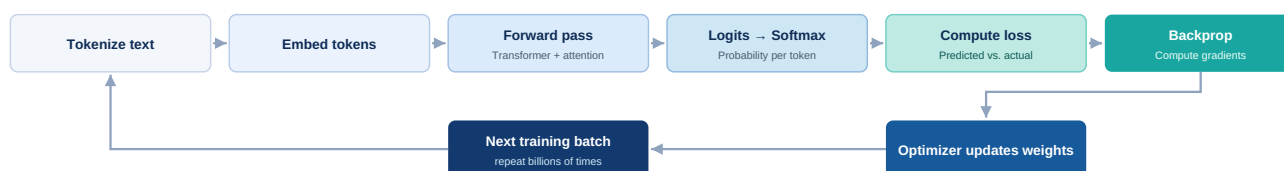


Figure 2a. One training step, end to end. The model's "learning" is this loop — predict, measure error, adjust weights slightly, repeat — executed at a scale of trillions of tokens and hundreds of billions of parameters.

Pretraining: pattern learning at enormous scale

In pretraining, the model repeatedly predicts the next piece of text in a document and is corrected when it is wrong — billions of times, across trillions of words. This single, simple mechanism, applied at scale on a **Transformer** architecture, is what produces a model's grasp of language, code, facts, and reasoning patterns. The Transformer's core innovation, **attention**, lets every word in a passage dynamically weigh every other word to resolve meaning — for example, working out what "it" refers to in "the bank approved the loan because it had sufficient collateral." This architecture was introduced in the 2017 paper that gave the field its now-famous title, *Attention Is All You Need*.^[1] Scaling research since has shown that a model's eventual quality is a predictable function of its parameter count, training data volume, and compute budget, jointly — not parameter count alone, which is why a well-trained smaller model routinely outperforms a larger, under-trained one.^[2]

A pretrained model is not yet a well-behaved assistant. It has absorbed language, facts, and patterns, but nothing has taught it to follow instructions, decline unsafe requests, or format an answer the way a business user expects.

Post-training: turning raw capability into a usable assistant

A second training stage — **instruction tuning** and **reinforcement learning from human feedback (RLHF)** — teaches the model to follow instructions, prefer helpful and honest answers, and refuse unsafe ones. This is the step that converts a raw pattern-completion engine into something resembling a professional assistant. The approach was formalized in OpenAI's InstructGPT research, which showed that a much smaller, human-feedback-tuned model was preferred by users over a larger, untuned one.^[3]

WHY THE TERM "FOUNDATION MODEL" EXISTS

Stanford's Center for Research on Foundation Models coined the term in 2021 to describe models "trained on broad data at scale" that can be adapted to a wide range of downstream tasks — language, vision, code — rather than built for one narrow purpose.^[4] That adaptability is precisely why one model can power a chatbot, a document summarizer, and an autonomous agent, without being retrained for each.

WHAT THIS MEANS FOR LEADERSHIP

Training a frontier foundation model from scratch is a capital-scale undertaking reserved for a handful of labs. The economically relevant decision for nearly every enterprise is not "should we train a model" but "which model do we build on, and how do we ground and govern it" — the subject of Sections 3–7.

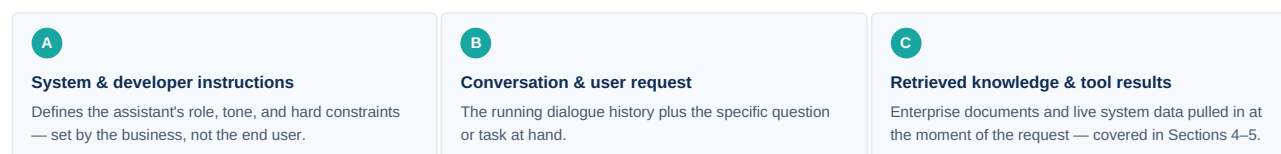
3. From Trained Model to Live Application

Training happens once, ahead of time. Inference happens every single time a user asks a question — and that distinction drives cost, latency, and architecture decisions.

Once a model is trained, using it is called **inference**: text goes in, a response comes out. Internally, input text is broken into **tokens** (word fragments), each token is converted into a numeric **embedding** that captures meaning, and the model's attention layers compute a probability distribution over what token should come next — one token at a time, feeding each choice back in as context for the next. This is why LLMs are described as **autoregressive generators**: they build an answer one predicted token after another.

What actually reaches the model

A live application rarely sends only the user's question to the model. It assembles a full **context**: system instructions that define the assistant's role and rules, the conversation history, the user's request, and — increasingly — retrieved documents and tool results (Sections 4 and 5). All of this is assembled into a single prompt before the model ever begins generating a response.



Where "reasoning" fits in

Some newer models are trained to allocate extra computation to harder problems — working through intermediate steps before producing a final answer, which improves performance on multi-step tasks. It is more precise to describe this as **learned, computational inference that exhibits reasoning-like behavior** than to equate it with human reasoning: the model is still fundamentally predicting likely next tokens, informed by patterns learned in training, not running a symbolic logic engine.

A GOVERNANCE-RELEVANT NUANCE

Because inference is probabilistic token prediction rather than database lookup, a model can produce fluent, confident, and incorrect statements — commonly called **hallucination**. This is precisely the problem that grounding a model in real enterprise data (Section 4) and validating its outputs (Section 9) are designed to control.

The distinction underneath everything: creation vs. usage

Sections 2 and 3, side by side, are the single most useful distinction in this entire document. Confusing the two is the single most common source of unrealistic AI timelines and budgets.

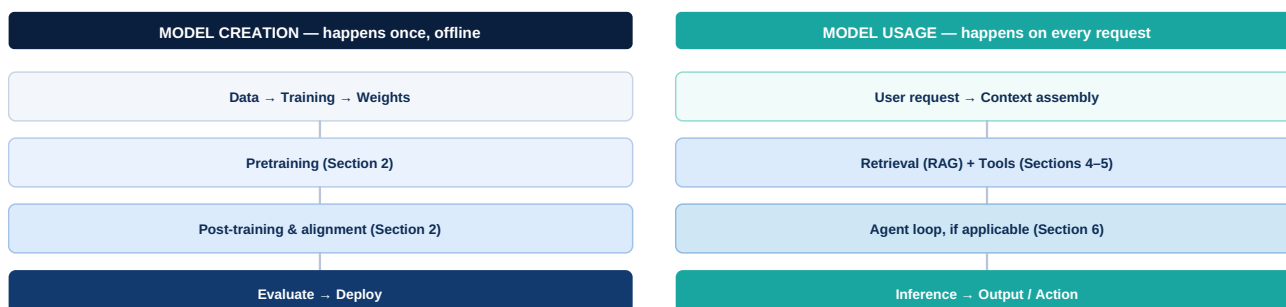


Figure 2b. Model creation is a capital-intensive project run occasionally, by a handful of labs. Model usage is a per-request operation every enterprise runs continuously — and it is where nearly all enterprise AI engineering effort actually goes.

WHAT THIS MEANS FOR LEADERSHIP

Every inference call has a marginal cost and latency, unlike a cached database lookup. Architecture choices — how much context is assembled, how many tool or retrieval round-trips occur, whether a smaller or larger model is used for a given step — directly drive the unit economics of an AI feature at production scale.

SECTION 4 · RETRIEVAL-AUGMENTED GENERATION

4. RAG: Grounding AI in Enterprise Knowledge

A model's training data has a cutoff date and knows nothing about your company. Retrieval closes that gap — without the cost or risk of retraining.

Ask a general-purpose model "what is our company's leave policy," and it will either decline or, worse, invent a plausible-sounding answer.

Retrieval-Augmented Generation (RAG) solves this by fetching relevant internal documents at the moment of the question and inserting them into the model's context before it answers — giving it access to current, proprietary knowledge without the model itself ever being retrained. The technique was introduced in a 2020 paper from Meta AI Research and collaborators, which combined a pretrained retriever with a pretrained generator into a single system.^[5]

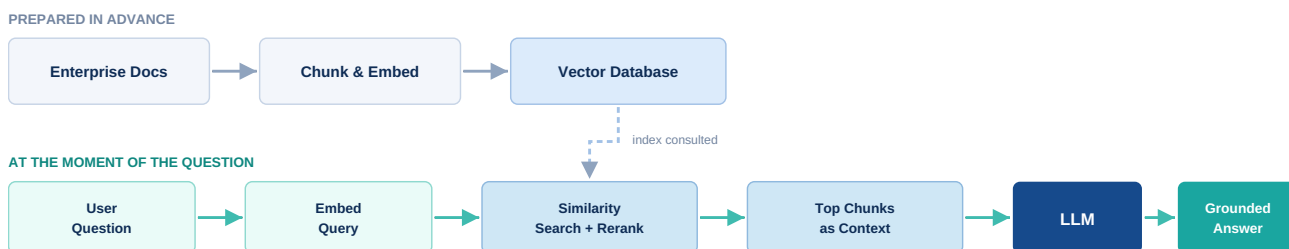


Figure 3. RAG in two phases: documents are indexed once in advance, then relevant chunks are retrieved fresh for every question and handed to the model as context.

Why this matters more than it sounds

APPROACH	WHAT IT DOES	TRADE-OFF
Retraining / fine-tuning	Bakes new knowledge into the model's parameters	Expensive, slow, and the knowledge still goes stale the moment new documents appear
RAG	Fetches current documents at question time and hands them to the model	Fast to update, source-attributable, but only as good as retrieval quality and document hygiene
Long-context prompting	Pastes large volumes of text directly into the prompt	Simple for small corpora; costly and slower to scale to enterprise-wide knowledge bases

Grounding vs. fine-tuning: different problems, different solutions

NEED	BETTER APPROACH
Current enterprise knowledge	RAG
Frequently changing information	RAG
New factual knowledge generally	Usually RAG
Domain-specific behavior or terminology	Fine-tuning
Output style, tone, or format	Fine-tuning or prompting
Specialized task behavior	Fine-tuning
Both current knowledge and specialized behavior	RAG + fine-tuning together may be appropriate

WHAT THIS MEANS FOR LEADERSHIP

RAG shifts the hard engineering problem from "train a smarter model" to "curate, structure, and keep current a trustworthy knowledge base" — a data-quality and information-governance challenge as much as an AI one. Enterprises that treat their document hygiene as a prerequisite investment consistently see better RAG outcomes than those that treat it as an afterthought.

5. Tool Use: Connecting AI to Systems of Record

Retrieval gives a model knowledge. Tools give it hands — the ability to check a live price, query a database, or trigger a workflow.

Some questions cannot be answered from documents at all — “what is the current price of our product” requires a live system, not a knowledge base. **Tool use** lets a model recognize that it needs external data or action, call a defined function (an API, a database query, a calculator, an internal system), receive the result, and incorporate it into its response. This is the mechanism that turns a model from something that only talks into something that can check and, eventually, act.

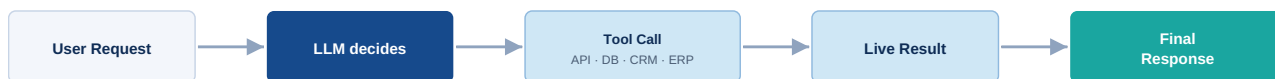


Figure 4. Tool use in one round trip. Agentic systems (Section 6) chain many such round trips together toward a larger goal.

Common enterprise tool categories

Systems of record

CRM, ERP, data warehouses, ticketing systems

Computation

Calculators, Python/SQL execution, analytics engines

Communication

Email, calendar, messaging, document generation

Infrastructure

Cloud provisioning, DevOps pipelines, monitoring

A CONTROL POINT, NOT JUST A CAPABILITY

Every tool a model can call is also a permission granted. Authentication, authorization, and scoping — which user is asking, what they are allowed to see, which tools they may trigger — belong in the orchestration layer, evaluated before a tool call executes, not left to the model's judgment alone.

WHAT THIS MEANS FOR LEADERSHIP

Tool integration work — not model selection — is usually the largest share of effort in a serious enterprise AI deployment. Budgeting and timelines should reflect that the connective tissue to existing systems, and its access controls, is the project.

6. Agents and Agentic AI: From Answers to Actions

The single biggest shift in the current wave of enterprise AI: systems that pursue a goal across multiple steps, rather than answer one question at a time.

Retrieval (Section 4) and tools (Section 5) both hand the model something to work with. An agent decides, across many such hand-offs, when to keep going. A conventional model interaction is a straight line: question in, answer out. An **agent** wraps a model in a loop — plan, act, observe, reason, repeat — that continues until the goal is satisfied or a human checkpoint is reached. This interleaving of reasoning and acting was formalized in 2022 research (widely credited as a founding pattern behind today's autonomous agent systems) showing that models prompted to alternate between reasoning traces and concrete actions solve tasks more accurately and more transparently than either alone.^[6] A related technique lets a model learn, largely on its own, which external tools to call and how to call them.^[7] Critically, **an agent is not a new kind of model**. It is a model combined with orchestration, tools, memory, and a control loop. The formula worth remembering:

LLM + orchestration + tools + memory/state + control loop = an agentic system.



Figure 5. The agent loop. Each pass may call one or more tools (Section 5) and consult retrieved knowledge (Section 4) before deciding whether another step is needed.

Agentic AI: from one agent to goal-directed systems

Agentic AI describes the broader paradigm: systems that pursue multi-step, goal-directed work with meaningful autonomy — “analyze our AI spending, compare it to benchmarks, and send recommendations to the CFO” — rather than single generation requests. A single agent can be fully agentic on its own; agentic AI does *not* require multiple agents. It simply describes goal-directed, iterative, partly autonomous behavior, whether one agent exhibits it or several. Complex work is often, though not necessarily, split across **multiple specialized agents** (research, analysis, risk, synthesis) coordinated by a master or orchestrating agent — a distinct pattern called **multi-agent orchestration**.

The twelve components of a production agent

“Wrap a model in a loop” is simple to say. A production-grade agent is actually assembled from distinct components, each a separate design and governance decision:

1–3 Goal, Planner, Model

The objective, the component that breaks it into steps, and the foundation model doing the reasoning

4–6 State, Memory, Tool registry

What the agent currently knows, what it recalls from earlier, and which tools it is permitted to call

7–9 Policy, Execution, Observation

Rules on what’s allowed, the layer that actually runs actions, and how results are read back in

10–11 Evaluator, Stop condition

Judges whether the goal is met, and the explicit rule that ends the loop — success, failure, or iteration limit — so it cannot run indefinitely

12 Human checkpoint

The point, if any, where a person must approve before the agent proceeds or acts

Workflow automation vs. AI agent

Not everything marketed as “agentic” needs to be. A conventional deterministic workflow is often the right, cheaper, more auditable choice:

TRADITIONAL WORKFLOW AUTOMATION	AI AGENT
Predefined, fixed path	Dynamic path, decided step by step
Deterministic execution	Probabilistic, model-driven execution
Fixed, hand-coded rules	Model-driven decisions from learned patterns
Known, structured inputs	Ambiguous, natural-language inputs
Fixed tool sequence	Dynamic tool selection per step
Highly predictable behavior	Requires ongoing evaluation and monitoring
Explicit branching logic	Adaptive, model-generated planning

A PRACTICAL RULE OF THUMB

If the steps and branches can be fully enumerated in advance, a traditional workflow beats an agent on cost, speed, and auditability. Reach for an agent only when the path genuinely cannot be known ahead of time. And once you do: autonomy is a dial, not a switch. Low-stakes, reversible tasks tolerate full autonomy well; high-stakes or hard-to-reverse actions (external comms, moving money, production changes) warrant a human checkpoint before execution — a deliberate design choice, not a default.

WHAT THIS MEANS FOR LEADERSHIP

Evaluate agent proposals by their loop, not their demo — the plan, the tools, the stop conditions, and exactly where a human signs off.

7. One Request, Traced Through the Full Stack

Every layer covered so far, working together on a single, realistic enterprise request.

Every layer from Sections 1–6 now comes together in one example. Consider a leadership request to an enterprise AI assistant: *"Analyze our Q2 sales performance, identify the three biggest problems, compare them with last year, and prepare a presentation for the leadership meeting."* Here is what happens between that sentence and a finished, reviewed deliverable.



Figure 6. The reference architecture underneath the walkthrough — every layer from Sections 1–6, plus the governance wrapper covered in Section 8.

WHAT THIS MEANS FOR LEADERSHIP

Notice how much of this sequence is *not* the model generating text: identity checks, data retrieval, calculation, validation, and human review. A realistic AI budget and timeline should weight these steps accordingly — they are usually the majority of the engineering effort, not the model call itself.

8. Governance, Risk, and Control for CXOs

Gartner projects 40% of enterprise applications will embed task-specific agents by end-2026^[9] — and separately warns that over 40% of agentic AI projects will be cancelled by 2027 over cost, unclear value, or weak risk controls.^[10] Governance is what separates the two outcomes.

Every layer covered in this document introduces its own risk surface. The following is not an exhaustive compliance checklist, but the set of questions a CXO should be able to get a straight answer to before funding an AI initiative past pilot stage.

RISK AREA	WHAT CAN GO WRONG	PRIMARY CONTROL
Data grounding	Model answers confidently from stale, wrong, or ungoverned documents	Curated, access-controlled knowledge base; source citations in every RAG answer
Hallucination	Fluent but factually incorrect output presented as fact	Grounding (Section 4), output validation, confidence signaling to the user
Tool & data access	A model triggers an action or exposes data the requester should not have	Least-privilege scoping evaluated before every tool call, not left to model judgment
Runaway agent loops	Multi-step agent repeats, stalls, or takes escalating unintended actions	Iteration limits, cost ceilings, explicit stop conditions, human checkpoints
Irreversible actions	Sending external comms, moving funds, altering production systems	Mandatory human-in-the-loop approval before execution
Cost & latency	Unbounded inference, retrieval, or tool-call volume inflates unit economics	Per-request budgets, model-tiering by task complexity, observability dashboards
Confidentiality	Sensitive fields surfaced to an unauthorized audience in a generated output	Output screening before delivery; audience-aware redaction rules

Human-in-the-loop is a design decision, not a default

A well-governed system draws an explicit line between actions an agent may take autonomously and actions that require sign-off. Gartner's own research projects human-in-the-loop involvement in IT operations workflows falling from roughly 95% in 2025 to about 40% by 2028 as trust and controls mature^[11] — a trajectory, not a starting point. The starting point for most high-stakes workflows should assume a checkpoint, then remove it deliberately as evidence accumulates.

WHY 40%+ OF AGENTIC PROJECTS STALL

Gartner's own analysis attributes cancelled agentic AI projects primarily to escalating costs, unclear business value, and inadequate risk controls^[10] — not to model quality. Each is addressable at the architecture and governance stage covered in this document, before a pilot is scaled.

WHAT THIS MEANS FOR LEADERSHIP

Request evaluation metrics, not just demos, before scaling any pilot: accuracy on a representative task set, cost per completed task, and an explicit incident/rollback plan. Treat these as launch criteria on par with a traditional software release, not optional documentation.

9. Strategic Implications and a Practical Roadmap

Sequencing matters. Most durable enterprise AI programs climb this ladder rather than skip to the top.

STAGE	MATURITY LEVEL	WHAT IT LOOKS LIKE	PRIMARY LEADERSHIP FOCUS	EXIT CRITERIA
1	Assistive chat	General-purpose assistant used for drafting, summarizing, and Q&A on public knowledge	Usage guidelines, data-handling policy	Security & data-handling policy in place
2	Grounded copilot	RAG-connected assistant answering from curated, access-controlled enterprise knowledge	Knowledge base curation, source governance	Retrieval quality validated against real queries
3	Connected assistant	Tool use added — the assistant checks live systems, not just documents	Access control, scoping, integration investment	Tool access scoped and authorized per system
4	Single-task agent	Multi-step autonomy within one bounded workflow, with human checkpoints	Evaluation metrics, cost ceilings, rollback plans	Evaluation suite and rollback plan proven in production
5	Agentic AI at scale	Multiple coordinated agents executing cross-functional, goal-directed work	Enterprise-wide governance, interoperability standards, audit	Cross-system governance and observability mature

Gartner forecasts agentic AI could drive roughly 30% of enterprise application software revenue by 2035 in its best-case scenario, up from about 2% in 2025^[9] — a multi-year curve, not a single-year leap. Organizations that skip stages, particularly moving from Stage 1 directly to Stage 4 or 5 without curated knowledge or tested tool integrations, are disproportionately represented among the cancelled projects referenced in Section 8.

Four recommendations for the next 12 months

1

Fund the data layer first

RAG quality is bounded by document hygiene, access control, and metadata — treat this as the prerequisite investment, not an afterthought.

2

Pick one bounded, high-value workflow

Prove the full architecture — retrieval, tools, evaluation, human checkpoint — end to end on one workflow before expanding scope.

3

Make governance a launch criterion

Evaluation metrics, cost ceilings, and rollback plans belong in the launch checklist, not a follow-up workstream.

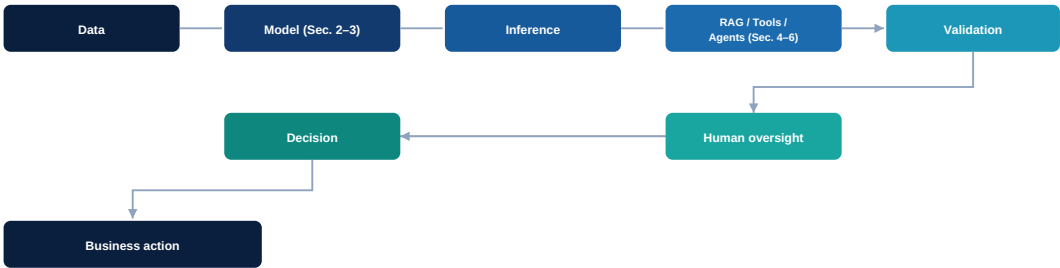
4

Build organizational fluency

Give teams the shared vocabulary in this document so that "AI," "GenAI," and "agent" stop being used interchangeably in planning conversations.

From data to decisions: the whole loop

Zooming back out to where this document started:



Governance (Section 8) wraps this entire loop, start to finish — not a step at the end.

Figure 7. The two journeys from Section 1, closed into one loop: data creates the model once; every request afterward runs inference forward through knowledge, tools, and agents, then back through validation and human oversight to a decision and an authorized action.

CLOSING THOUGHT

The technology in this document is best understood as one continuous pipeline from data to decision, wrapped in governance at every layer. The business process — and the judgment about which decisions an organization is willing to delegate, to what degree, under what oversight — is what ultimately separates "AI technology" from "enterprise AI" that reliably creates value.

APPENDIX A

Glossary

Plain-language definitions of every term used in this document.

Agent

A model combined with orchestration, tools, memory, and a control loop, able to pursue a goal across multiple steps.

Agentic AI

The broader paradigm of goal-directed AI systems that plan, act, and adapt with meaningful autonomy — whether via one agent or several.

Artificial intelligence (AI)

The broad field of building systems that perform tasks normally requiring human intelligence — reasoning, perception, language, decision-making.

Attention

The Transformer mechanism that lets each token weigh the relevance of every other token when building meaning.

Backpropagation

The algorithm that computes how much each parameter contributed to a training error, used to update the model.

Context window

The full set of text — instructions, history, retrieved data, tool results — assembled and sent to the model for a single inference call.

Decoding / sampling

Turning a model's token probabilities into an actual chosen next token — deterministically, or randomized within limits.

Deep learning

A subset of machine learning using layered neural networks; the approach behind nearly all modern foundation models.

Embedding

A numeric vector representation of text (or other data) that captures learned semantic meaning.

Epoch

One complete pass through a training dataset during model training.

Fine-tuning

Further training a pretrained model on a narrower dataset to specialize its behavior for a specific task or domain.

Foundation model

A model trained on broad data at scale that can be adapted to many downstream tasks, rather than built for one narrow purpose.

Function / tool calling

The specific mechanism by which a model requests that a defined external function be executed, and receives its result.

Generative AI (GenAI)

Models and applications that produce new content — text, image, audio, video, or code — rather than only classifying or predicting.

Gradient

A measure of how much a small change in a parameter would change the training error, used to decide how to update it.

Grounding

Anchoring a model's output in verifiable external information, typically via RAG, rather than relying solely on training-time knowledge.

Guardrails

Automated checks — input filters, output validators, policy rules — that constrain what a model or agent is allowed to say or do.

Hallucination

A fluent, confident model output that is factually incorrect or unsupported by any source.

Human-in-the-loop (HITL)

A design pattern requiring human review or approval before a system's output or action is finalized.

Inference

The act of running a trained model to produce an output for a given input, as opposed to training the model.

Large language model (LLM)

A foundation model specialized in processing and generating human language.

Learning rate

A setting controlling how large a step the optimizer takes when updating parameters after each training batch.

Logits / softmax

Logits are a model's raw, unnormalized scores for each possible next token; softmax converts them into a probability distribution.

Loss function

The formula that scores how wrong a model's prediction was during training; training works to minimize this score.

Machine learning (ML)

An approach within AI where systems learn patterns from data rather than following hand-coded rules.

Memory (agent)

State an agent retains across steps or sessions — short-term working context or longer-term stored history and preferences.

Multi-agent orchestration

Coordinating several specialized agents (e.g. research, analysis, synthesis) under a master agent toward one larger goal.

Neural network

A computing system of layered, connected nodes ("neurons") whose connection strengths are learned from data.

Observability

Instrumentation and monitoring — logs, traces, metrics — that let a team see what an AI system did and why, after the fact.

Orchestration

The layer that coordinates a model's calls to tools, retrieval, memory, and other components toward completing a task.

Parameter / weight

A learned numeric value inside a model; the collective configuration of billions of parameters encodes what the model has learned.

Pretraining

The initial, large-scale training stage where a model learns language, facts, and patterns by predicting missing or next text.

Prompt

The text input given to a model at inference time, including instructions and the user's request.

Quantization

Reducing the numerical precision of a model's parameters to shrink its size and increase inference speed, usually with a small accuracy trade-off.

RAG (Retrieval-Augmented Generation)

Fetching relevant documents at question time and supplying them to the model as context, grounding its answer in current knowledge.

Reranking

A second-pass scoring step in retrieval that reorders initially retrieved documents by relevance before they reach the model.

RLHF

Reinforcement Learning from Human Feedback — a post-training technique that aligns a model's behavior with human preferences.

Scaling laws

Empirical relationships showing how model quality improves predictably with more parameters, data, and compute, used together.

Semantic / hybrid search

Retrieval that matches meaning via embeddings rather than exact keywords; hybrid search combines semantic and keyword matching.

System prompt

Instructions set by the application, not the end user, that define an assistant's role, tone, and hard constraints for every request.

Temperature (decoding)

A setting controlling how random or deterministic token selection is during decoding; lower values produce more predictable output.

Tokenization

Splitting text into sub-word units ("tokens") that a model processes numerically.

Tool use

A model's ability to call an external function — an API, database, or calculator — and use the result in its response.

Transformer

The neural network architecture, built around attention, underlying nearly all modern foundation models.

Vector database

A database optimized to store embeddings and quickly find the most similar vectors to a given query — the retrieval engine behind RAG.

APPENDIX B

References

Primary, independently verifiable sources cited throughout this document, grouped by subject.

Foundational architecture & training

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). [Attention Is All You Need](#). NeurIPS 2017. arXiv:1706.03762
2. Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., et al. (2022). [Training Compute-Optimal Large Language Models](#) (Chinchilla). NeurIPS 2022, DeepMind. arXiv:2203.15556
3. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., et al. (2022). [Training Language Models to Follow Instructions with Human Feedback](#). NeurIPS 2022. arXiv:2203.02155
4. Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., et al. (2021). [On the Opportunities and Risks of Foundation Models](#). Stanford Center for Research on Foundation Models (CRFM). arXiv:2108.07258

Retrieval & grounding

5. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#). NeurIPS 2020, Meta AI Research. [proceedings.neurips.cc — NeurIPS 2020 paper](#)

Tool use & agents

6. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). [ReAct: Synergizing Reasoning and Acting in Language Models](#). arXiv:2210.03629
7. Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). [Toolformer: Language Models Can Teach Themselves to Use Tools](#). arXiv:2302.04761

Governance, risk & industry outlook

8. National Institute of Standards and Technology (2023, January 26). [Artificial Intelligence Risk Management Framework \(AI RMF 1.0\)](#). NIST AI 100-1, U.S. Department of Commerce. [nist.gov/itl/ai-risk-management-framework](#)
9. Gartner (2025, August 26). [Gartner Predicts 40% of Enterprise Apps Will Feature Task-Specific AI Agents by 2026, Up from Less Than 5% in 2025](#). Press release. [gartner.com/en/newsroom/press-releases](#)
10. Gartner (2025, June 25). [Gartner Predicts Over 40% of Agentic AI Projects Will Be Cancelled by End of 2027](#). Press release, Gartner IT Symposium/Xpo. [gartner.com/en/newsroom/press-releases](#)
11. Gartner (2025, December 4). [Predicts 2026: AI Agents Will Transform IT Infrastructure and Operations](#). Gartner Research (Haight, C., Banerjee, A., Antelmi, J., Forest, J., Mukhyala, C.). Cited via Itential analyst-report summary

A NOTE ON SOURCING

Every reference above links to the original paper, publisher, or press release rather than a secondary summary, so leadership teams and their staff can verify each claim directly. Gartner citations reflect published forecasts as of their release dates and should be checked against Gartner's current research for any subsequent updates.